

CS 1650 Project Report

Cory Zimmerman, Fall 2024

Project repo: <https://code.harvard.edu/coz849/cs165-project/tree/master>

Intro

These tests focus on the performance and robustness of various implementations in my database. Because memory access was a primary theme of the course, most tests use data on page faults to explore performance characteristics. All my tests were automated via Python scripts, and results are averaged over multiple trials. I also experimented with cachegrind, but it was more difficult to automate and didn't seem to track disk accesses. Cache info from Perf on M1 Mac Docker seems to be unsupported.

I developed my project and ran all experiments within the CS 165 docker container on a 2021 M1 MacBook Pro. When tests were running, Docker, a terminal, and my database were the only GUI applications in use (although machine processes were probably running elsewhere). I turned off Bluetooth and made every effort to ensure a consistent testing environment.

If these tests share a theme, it's that the concepts we discussed in class are empirically valid. Despite all the possible variance in my implementations, testing scripts, host machine, etc., some results came out almost exactly as discussed in class. This was especially true in the Milestone 3 experiments. In M3E1, index performance versus table scan performance looks extremely similar to the graph we discussed in class, and my smooth scan cutoff is surprisingly close to the 10 percent magic number. Similarly, M3E3 showed that B+ Trees really are worth the complexity over binary search on large data sizes due to cache friendliness. There are other examples as well, but the experiments overall provided a powerful validation of the concepts I've learned this semester.

Data and graphs

All my results were generated by script and written into a CSV. I imported those CSVs into Google Sheets to make graphs. The raw data for every graph is available at this link: <https://docs.google.com/spreadsheets/d/19D8iLppTcAcokcOGRl3WwG9J3LNSlrV1pTi-Dl4HWc4/edit?usp=sharing>

Milestone 1

Experiment 1: How does data size affect scan performance?

Setup

Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m1e1.py>

The goal of this test is to determine how my select and fetch operators scale as column length increases.

I made a Python script that generates loadable csvs with two columns each. Each column has random data distributed over the range $[-\text{col_len} // 2, \text{col_len} // 2]$. For example, a column with 100 entries would have data entries distributed around $[-50, 50]$. I did this so that there's a consistent number of positive, negative, and duplicate values in every column:

<https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/gen.py>

I made another Python script that loads the csvs into the database. I generated and loaded tables with the following lengths (2x exponential growth maxing at 200 million): 390_625, 781_250, 1_562_500, 3_125_000, 6_250_000, 12_500_000, 25_000_000, 50_000_000, 100_000_000, 200_000_000:

<https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/load.py>

[Aside: I set 200 million as the maximum because my docker container was unable to handle more at the time of making these tests. My client previously handled load commands by reading the entire csv into memory and parsing it into chunks to be handled by the server, which limited loads to the maximum CSV size I could fit in memory. I've since updated the client to memory map the csv during parsing, which avoids this limitation.]

The test allocates and zeros 8 GB of memory to purge caches before starting the server. Then, for 5 trials of each size, it times how long a client takes to perform five selects and five fetches. The scans are interwoven, and each fetch retrieves the output of the select. Selectivity is 0.5%, and query ranges are randomly chosen. Output is written to a csv.

[Aside: This test is the only one that uses memory allocations to “purge” caches. All the others use the method described in M1E2. Allocations proved unreliable for dropping cached pages.]

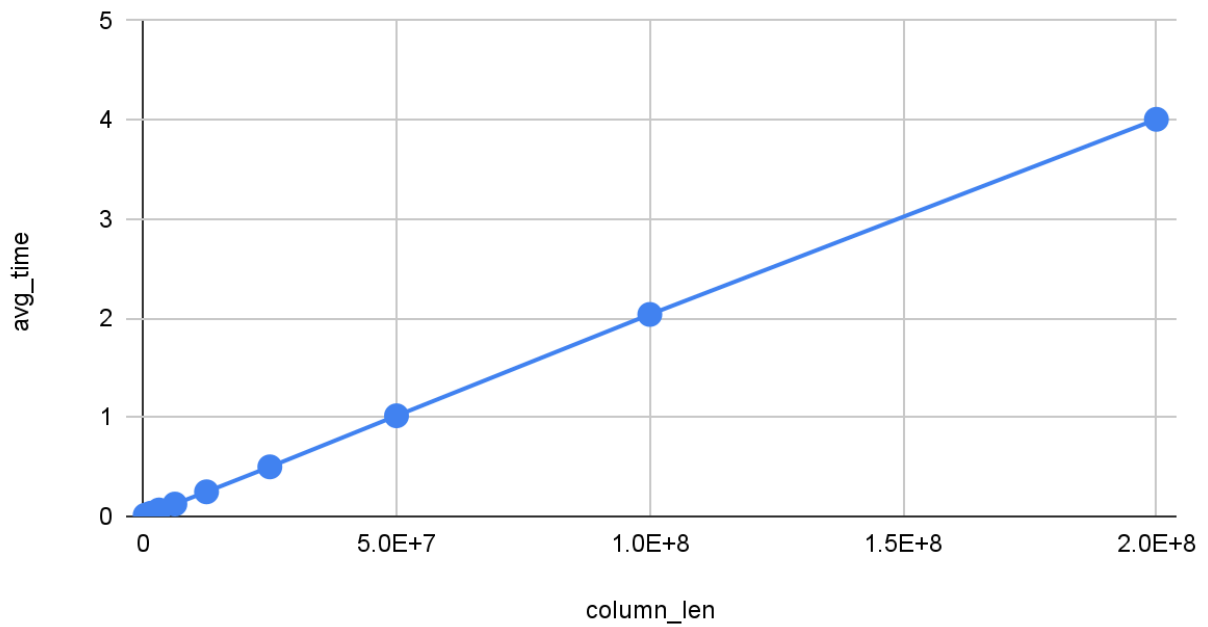
Select and fetch operations are likely dominated by disk accesses. Since `proc/pid/io` doesn't count mmap disk reads, I used page faults as a proxy for how often memory is fetched from disk: `ps -o majflt --no-headers -p <server_pid>`. The explanation graph below shows how many major page faults occurred on average for each data length in the performance tests, which approximates how many times disk reads are requested.

[Edit: later tests use `ps -o maj_flt,min_flt --no-headers -p <server_pid>` because different experiments are sensitive to different layers of the memory hierarchy and therefore different kinds of page faults.]

The resulting data tells how long it takes to start a client, send the query, execute five select + fetch pairs, and then exit the client for various data sizes.

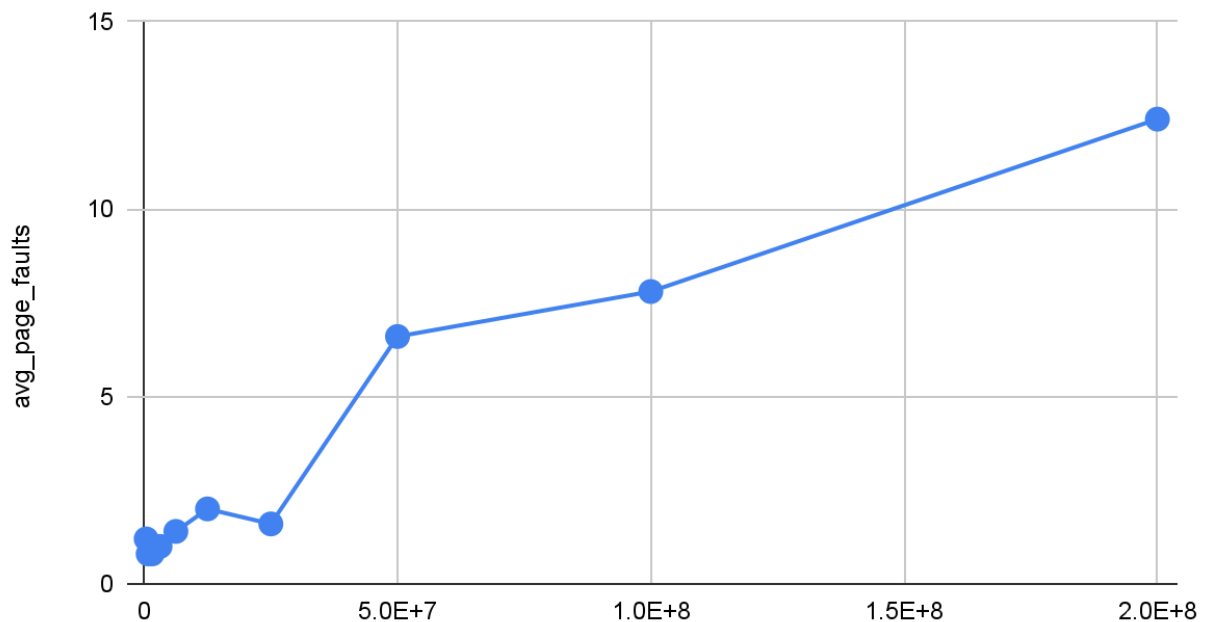
Result graph

Average query time



Explanation graph

Average page faults



Discussion of the result graph

The result graph shows query execution time growing linearly as column length increases, and the rate of growth is neither impressive nor alarming to me. Growth of this shape is expected since the test mostly runs linear scans. The steadiness of the curve indicates scan robustness and suggests that the select and fetch operators don't have any serious degeneration as column size grows.

In the linked spreadsheet, I also compute a "microseconds/column_len" value, which further demonstrates that scan performance is roughly constant per tuple as data size increases.

Discussion of the explanation graph

The major page fault graph is less steady but shows a similar shape. The third-to-last test (50 million) is the point at which major page faults spike. Because the test runs (select colA, fetch colB) five times for each column size, I suspect 50 million is the length at which different runs of the same test begin evicting each other from main memory. The linear growth rate after 50 million roughly matches that of the runtime curve, indicating a performance trend that should continue as column lengths scale.

Experiment 2: How do select and fetch performance scale as selectivity increases?

Setup

Script:

<https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m1e2.py>

Much class time and effort has been dedicated to improving selection speeds, but I've spent relatively little effort optimizing fetches. My goal in this experiment is to explore the performance impact of fetches relative to selections as the amount of selected data grows.

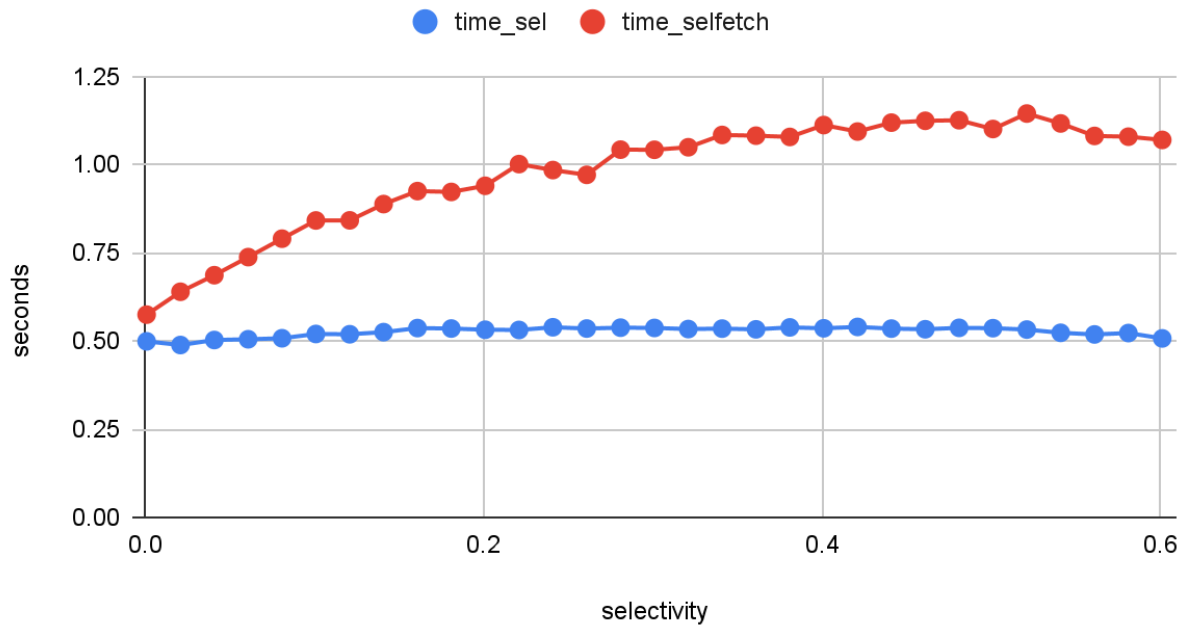
This test used much of the same infrastructure as M1E1, but my previous approach at cache purging was ineffective. Now, every trial purges caches with `echo 3 > /proc/sys/vm/drop_caches`, which frees "pagecache, dentries and inodes" ([source](#)). This affects every process running in the container including the client and test script, but server performance generally seems to dominate results after this command. My intention is to prevent caching between trials of the same test from skewing results. After purging caches, results for each trial should more consistently show the cost of running a query on a fresh DB server.

For each selectivity in the range (selectivity = 0.001, selectivity < 0.61, selectivity += 0.02), the test starts the server, purges caches, runs 20 select(colA) queries, collects stats, restarts the server, purges caches, runs 20 select(colA) fetch(colA) pairs, and collects stats. Results are averaged over five trials for each configuration. The runtime and page fault difference between select and select + fetch is assumed to be the performance profile of fetching. Because fetches occur on a column that was recently scanned, they aren't unfairly penalized for accessing an extra column.

[Aside: In my understanding, major page faults indicate attempts to access a memory mapped address that's not yet in main memory and needs to be retrieved from disk. This makes it a suitable proxy for cache misses on disk pages. In contrast, minor page faults occur when accessing a memory address that doesn't have a physical page in memory. This makes minor page faults a signal for frequent, large, and/or thrashed heap allocations that force the runtime to frequently assign or load new physical pages to requested addresses. Therefore, minor page faults are a rough but approximate measure for main memory cache friendliness.]

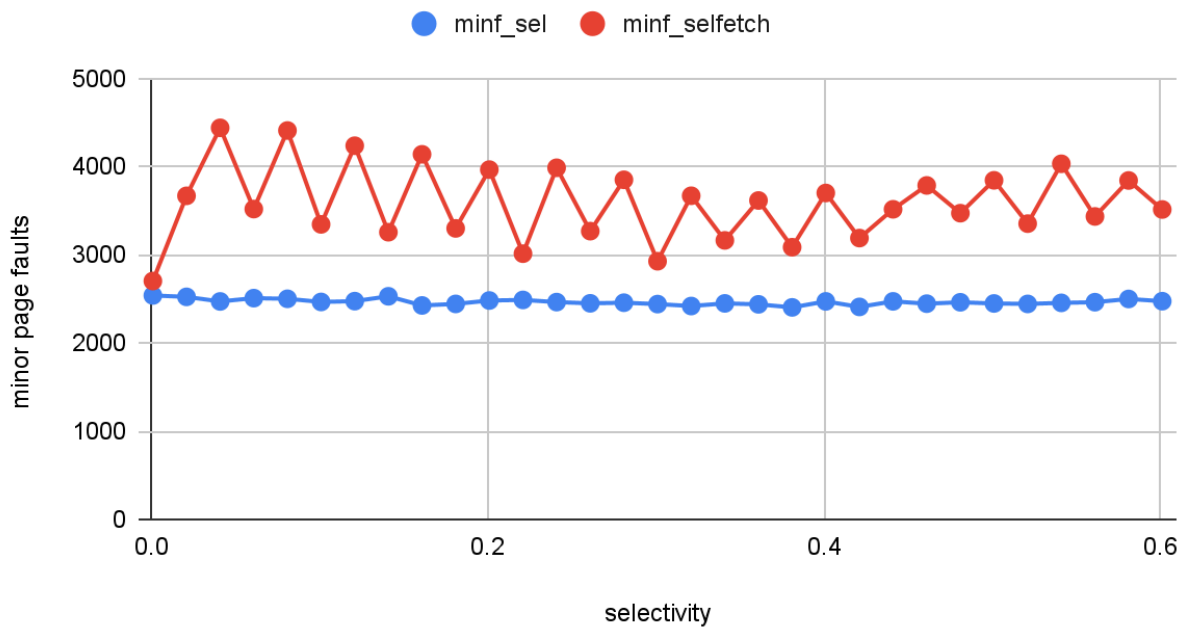
Result graph

Select v. select + fetch: runtime by selectivity



Explanation graph

Select v. select + fetch: minor page faults



Discussion of the result graph

At selectivity below about five percent, fetch adds little enough overhead that selection runtime dominates. Fetch runtime grows with selectivity until it equals select at about 20 percent selectivity. This likely reflects my underinvestment in fetch; it's not multithreaded, and I haven't tuned it as much. For the low selectivity queries we've typically tested, this isn't much of an issue, and the results here indicate that fetch only becomes a priority if query selectivity frequently exceeds five percent.

Selection performance is basically flat as selectivity increases. The select operator in this test uses bitvecs, so there are no changes to select's memory allocations or computation as selectivity increases – it just writes more ones instead of zeros into the bitvec.

Discussion of the explanation graph

This graph is more clue than explanation. I've done these out of order, so this is my second to last experiment. On all previous experiments, the outputs are extremely steady between runs. I attribute that to a relatively consistent machine state and averaging results across multiple trials. However, the minor page fault results for this test were extremely irregular between runs. This was my fourth attempt generating results (takes about 10 minutes to run all the trials in this test), and minor page faults make no real sense considering the changing workload. Major page faults are about the same between select and fetch, which matches expectations because they scan the same 12.5 million tuple column.

Because every other experiment has yielded extremely consistent results, I'm concluding that this output is a valid reflection of my database performance. With that in mind, it likely also indicates a serious issue with the main memory friendliness of fetches.

I suspect that large fetch outputs have a substantial impact on memory allocations. In a test like this, I'm making huge and frequent malloc and free calls to store and release all this fetched output. Growing mallocs likely show up as fairly smooth runtime growth, but internally I'm guessing it thrashes some layer of memory. While the issue certainly could be something else, this was the best explanation I could arrive at for why minor page faults here look so cursed compared to every other experiment.

The minor page faults here are so weird that I considered not even including this output in the report. However, I do think it's an honest indication of some performance bug in my DB. I've reviewed the code and don't see any immediate concerns. If the reader has any ideas about what's going on here, I'd really appreciate feedback.

Bitvec fetch operator:

<https://code.harvard.edu/coz849/cs165-project/blob/7a7182da38854f2e73691e920c4d675232eb348b/src/operators.c#L299>

Milestone 2

Experiment 1: How does the number of concurrent queries affect select performance?

Setup

Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m2e1.py>

The goal was to examine how runtime performance per query scales as more select queries are added to a single shared scan.

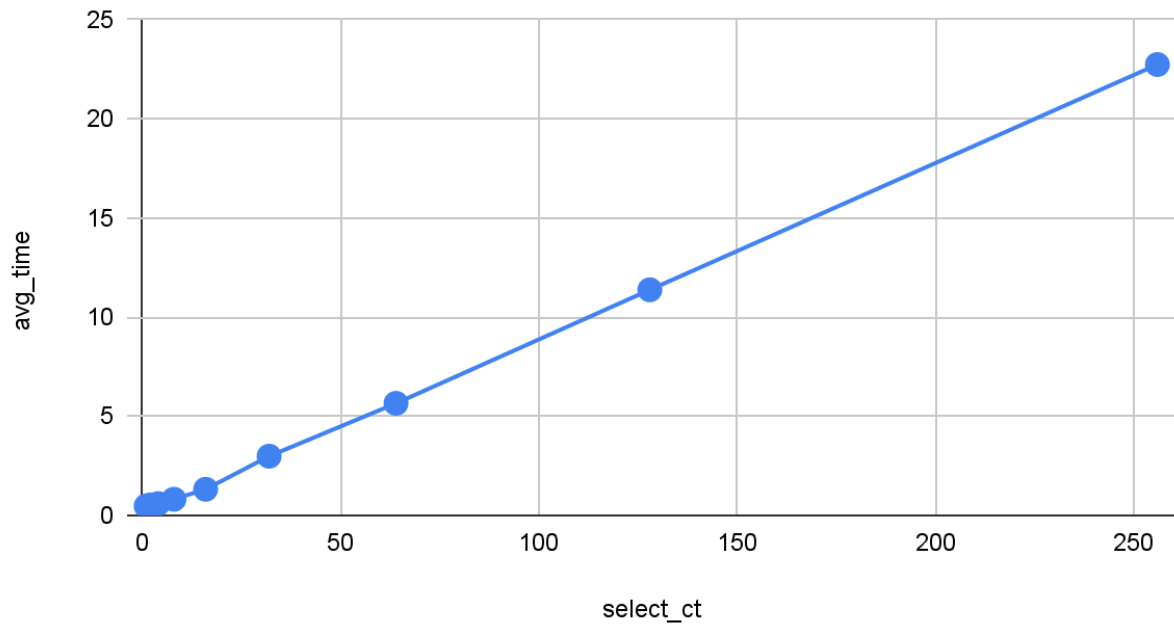
This test uses much of the same infrastructure as M1E2. It brings up the server, runs the query, counts runtime and page faults, and then terminates the server. This is done for each power of two number of queries from 1 to 256, and results are averaged over five runs for consistency. All scans occur over the table with 100 million tuples, and select ranges are randomly distributed with 0.5% selectivity.

Inputs look like this:

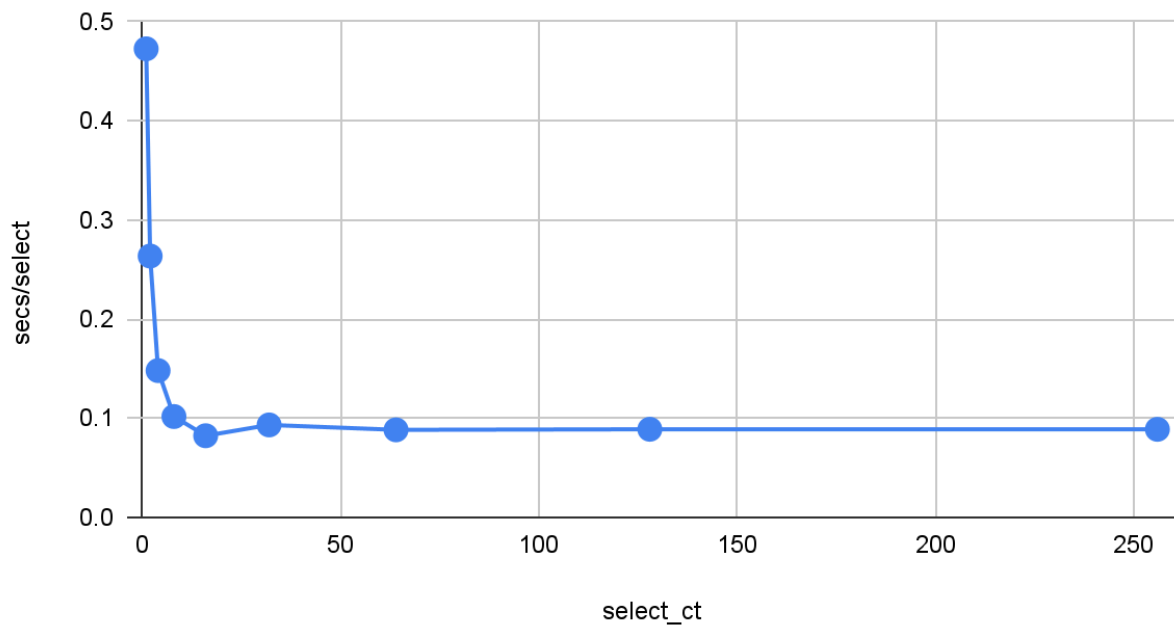
```
...
batch_select()
s0 = select(col0, min0, max0)
***
sN = select(col0, minN, maxN)
batch_execute()
...
```


Result graphs

Average time (total)

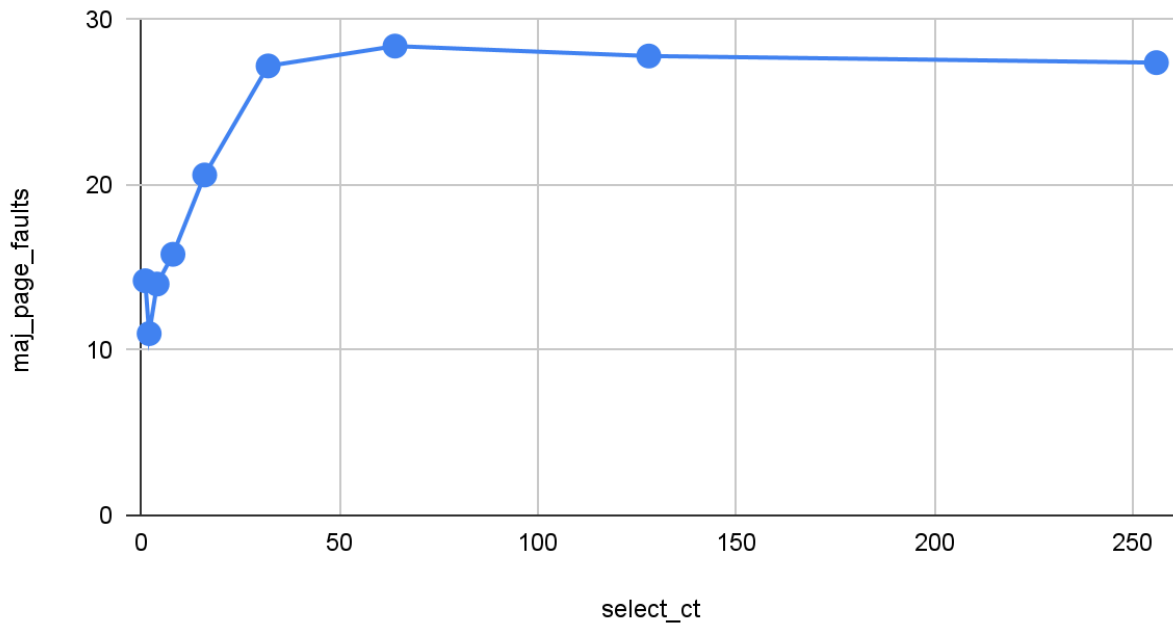


Seconds per select

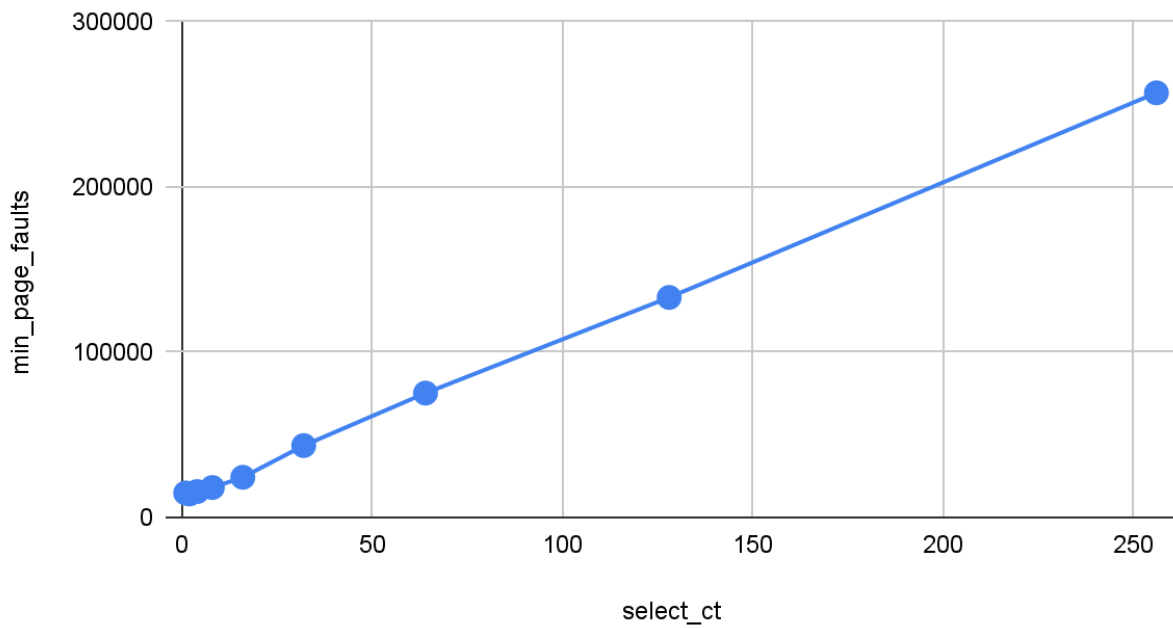


Explanation graph

Average major page faults



Average minor page faults



Discussion of the result graphs

These graphs show the total time to perform a single batch selection with N select queries over a column with 100 million entries. The total time increases linearly. While Google Sheets normalizes the graph scale, the rate of growth is also quite moderate. This result is best explained by the *seconds per select* graph, though, which shows that the total time amortized over all select queries is much better than a single query.

I'm surprised that the batch performance never really improves past what it reaches at eight batched selections, which I explore more below. Alone, the flat line seemingly discredits using higher counts of batch selection. However, in a production system, batching 256 queries together could save a lot of network time, in which case performance similar to eight batched queries on 256 batched queries saves 32 extra round trips.

Discussion of the explanation graph

The batch select algorithm runs one multicore scan and updates each query's result for each integer in the column. We also discussed in class how batch sizes should be limited to the TLB's size. In this case, though, major page faults spike at 16-32 selects per batch and level off until the end (256 selects per batch). My result bitvecs take up a substantial amount of space, and I suspect that page faults rise because of bitvecs pushing cached DB pages out of main memory.

Minor page faults are interesting because they mirror the growth in query execution time. I suspect this is caused by my DB using lots of `malloc` memory to store bitvec results and frequently accessing unassigned new results pages. This seems correlated with TLB misses as well, although I couldn't find a good way to track those without perf. Overall, I suspect that non-disk memory gets thrashed heavily as more select operators are added to a batch. If I needed to rewrite my shared scans to re-parse batch selections into smaller chunks, it looks like eight at a time is best on my Mac.

Experiment 2: How effectively does multithreading improve select performance?

Setup

Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m2e2.py>

This experiment tests how well my multithreaded select operator uses the CPUs available to it.

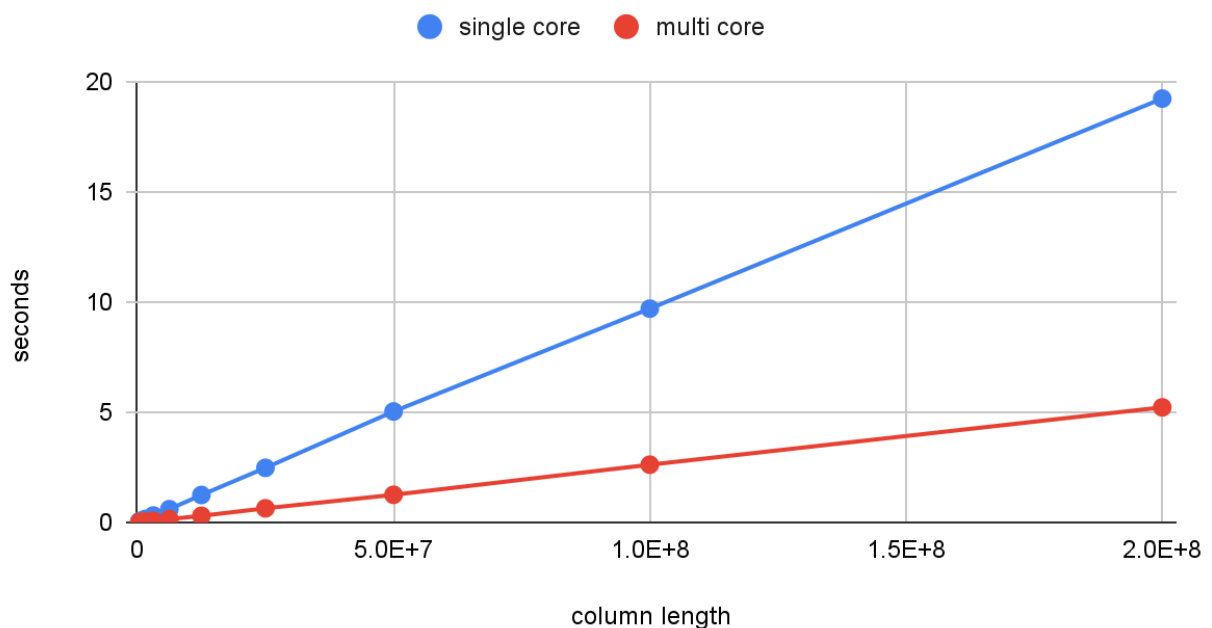
For each of (single core, multi core), the test starts a server and runs five trials of 48 batched selection queries for each table size. Average runtime and maximum percent CPU utilization are logged to a CSV. Batch and normal selection use the same operator, so I chose batching to waste less time on I/O and run a higher scan workload.

Single threaded tests use `taskset -c 3 /cs165/src/server` to start the db server pinned to core 3 making it effectively single threaded (core choice arbitrary). The second run starts the server with no limits. This is preferable to the `single_threaded` DSL queries because they restrict thread usage within a select operator but don't modify my threadpool, which still occupies more than a single core.

This test uses Python's async runtime. While the server is processing, the test runs a separate loop that polls `ps -o pcpu --no-headers -p <server_pid>` every 0.01 seconds and tracks the highest cpu utilization ever encountered across all trials for each configuration.

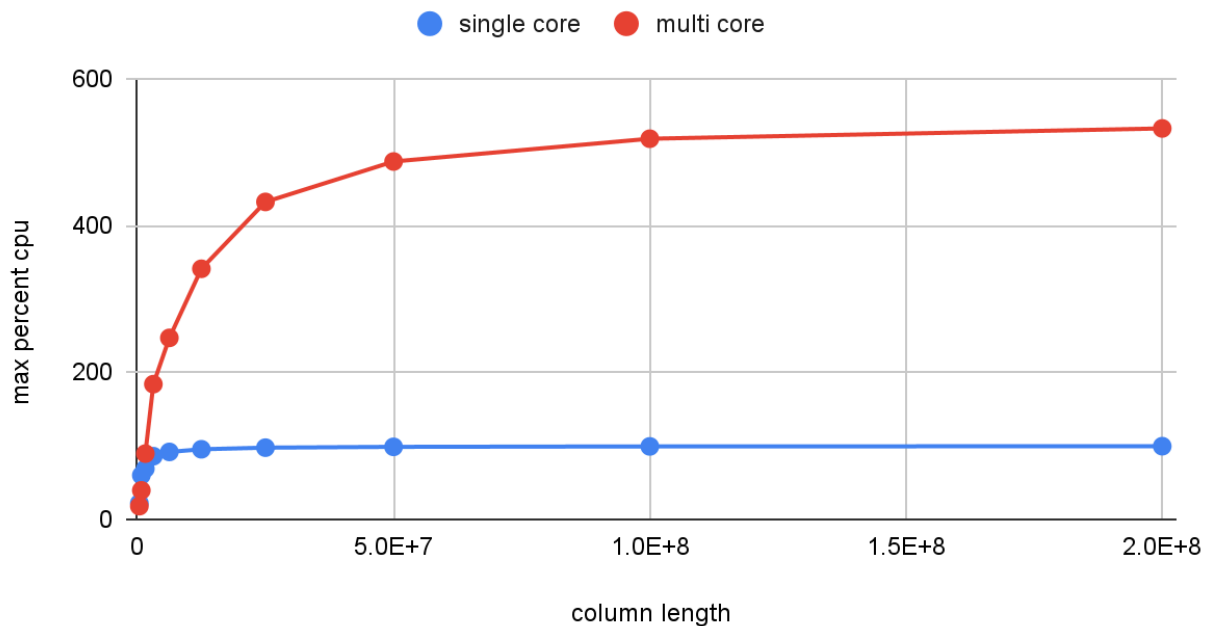
Result graph

Batch select runtime: single v. multi core



Explanation graph

Batch select max percent CPU: single v. multi core



Discussion of the result graph

Similar to M1E1, scan performance grows robustly as data size increases. As expected, the multi core variant grows at a substantially flatter rate than the single core version. At a column length of 200 million, my laptop completed the multi core algorithm almost 4x faster than the single core version (5.2 v. 19.2 seconds).

Discussion of the explanation graph

CPU utilization for the single core variant reaches 90 percent by a table length of 6.25 million and 99 percent by 100 million. This might reflect some irregularity in the 0.01 second polling interval for CPU usage, but it might also indicate smaller workloads not maximizing available resources for very long. The multithreaded version reaches 532 percent in this test, but a previous run of the script showed maximum utilization exceeding 600 percent. Considering my laptop is also handling docker, python, an async python runtime, and frequent calls to `ps` while running this benchmark, I consider ~530 percent CPU utilization decent on this 9-core thread pool (accounting for synchronization slowdown as well). It's certainly more compelling than the ~240 percent found in M4E2. My select algorithm also intentionally sacrifices some thread synchronization time for a cleaner memory access pattern, which makes less than maximum cpu utilization a bit more acceptable.

Milestone 3

Experiment 1: How does selectivity affect smooth scan performance?

Setup

Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m3e1.py>

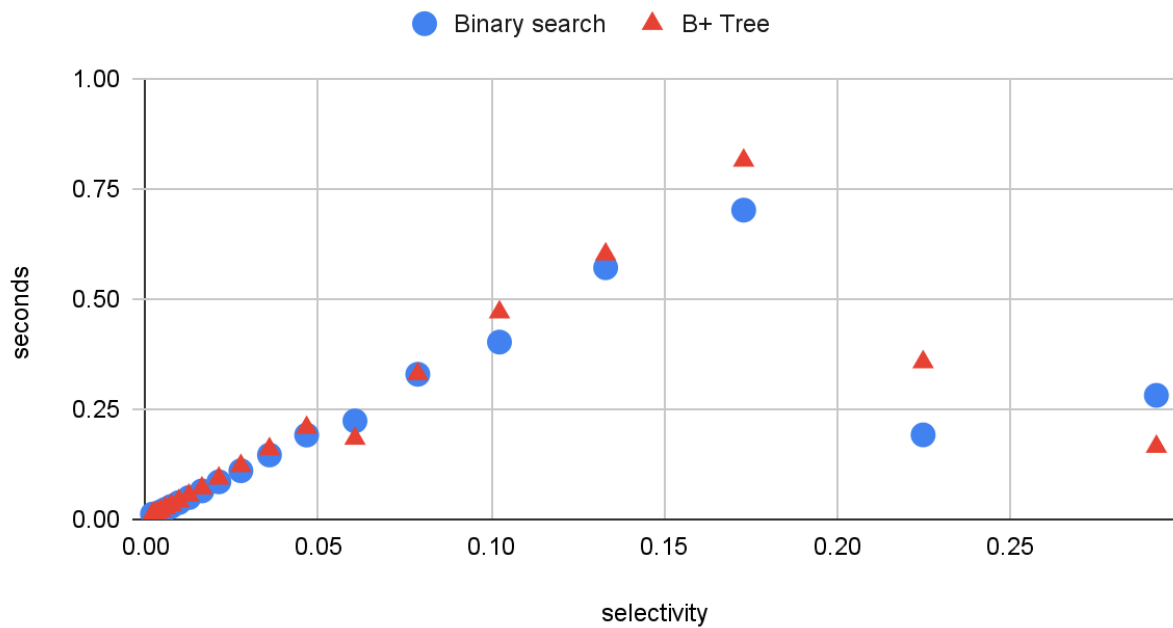
The goal of this test is to demonstrate how scan performance scales with selectivity and whether my smooth scan cutoff is effective.

For index analysis, I created a new db with a single table and three columns of length 50 million. Data distribution is the same as M1E1. Column 1 has a primary sorted index, column 2 has a secondary sorted index, and column 3 has a secondary btree index.

For each of the two secondary columns, I restart the server and sleep until the indexes are built. Then, for the range (selectivity = 0.002, selectivity < 0.36, selectivity *= 1.3), I collect and average results across five trials. Each trial purges caches like in M1E2, runs a select query, and counts page faults.

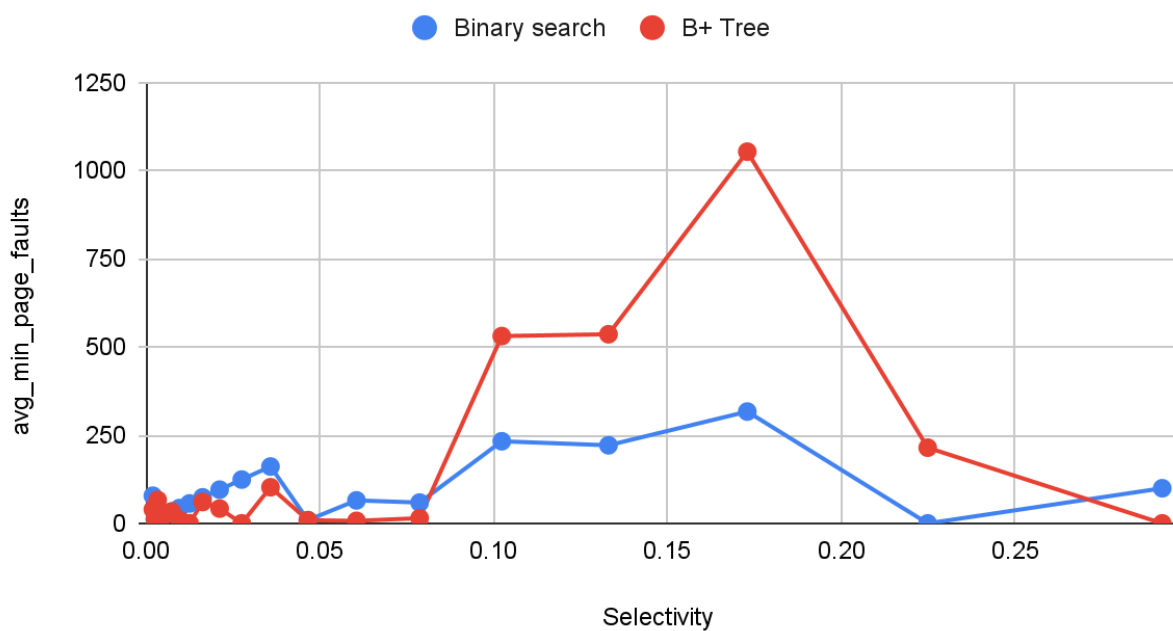
Result graph

Secondary index smooth scan: seconds/select



Explanation graph

Secondary index smooth scan: minor page faults/select



Discussion of the result graph

When this data was collected, my database used a cutoff of 20 percent estimated selectivity before switching from an index to a full scan. The graph shows runtimes climbing up to a peak, after which the fairly-steady 0.25 second full table scans take over.

Both binary search and B+ tree queries appear substantially faster than full column scans below about 5 percent selectivity. However, as selectivity grows past 10 percent, index performance degenerates rapidly, making a full table scan better. The decrease in B+ tree runtime from 22 to 29 percent selectivity is puzzling, but I suspect it's attributable to receiving comparably more or less favorable query ranges. At selectivity this high, such factors can have a serious impact.

The shape of these lines is expected from our discussion in class, but I expected my index cutoff to be higher than it is. With these results, I've lowered my database's smooth scan cutoff to 10 percent.

Discussion of the explanation graph

My method for cache purging only wipes files that aren't memory mapped. Because building the DB index itself introduces lots of page faults, I didn't see an easy way to consistently flush and measure major page faults. Purging should cause accurate minor page faults, so that's what I measured.

The results are very noisy, but minor page faults generally follow a similar pattern as runtime. There's a rising index line up to the peak, after which things switch to a full scan and level off. The spike in B+ tree faults matches my expectations. I use two different B+ trees in the database. Primary indexes use a very streamlined B+ tree that holds `uint64_t` values. Clustered selects use that tree to retrieve the start and end column positions of a query on the column. In contrast, secondary indexes hold vectors of values for each key so that leaves can be scanned as a linked list in place of a column scan. However, that means every key possibly includes a cache miss for the leaf and a cache miss for the vector's storage. Because vectors are larger than a `uint64_t`, the leaves also aren't as cache efficient. I profiled the trees separately, and the first B+ tree is about three times faster per traversal than the second one (used here).

If anything, these results validate secondary index B+ tree performance more than expected. Binary search is simple code, and copying results takes a single `memcpy`. The B+ tree is much more complex and has a linked-list style traversal. While the B+ tree degenerates more dramatically, it's still surprisingly competitive with binary search.

Clustered index B+ tree (tested in M3E3):

<https://code.harvard.edu/coz849/cs165-project/blob/master/src/include/bptree.h>

Unclustered index B+ tree (tested here):

<https://code.harvard.edu/coz849/cs165-project/blob/master/src/include/bptree2.h>

Tree comparison: https://code.harvard.edu/coz849/cs165-project/blob/master/src/perf_bptree.c

Experiment 2: How does a clustered sorted index affect scan performance as data size grows?

Setup

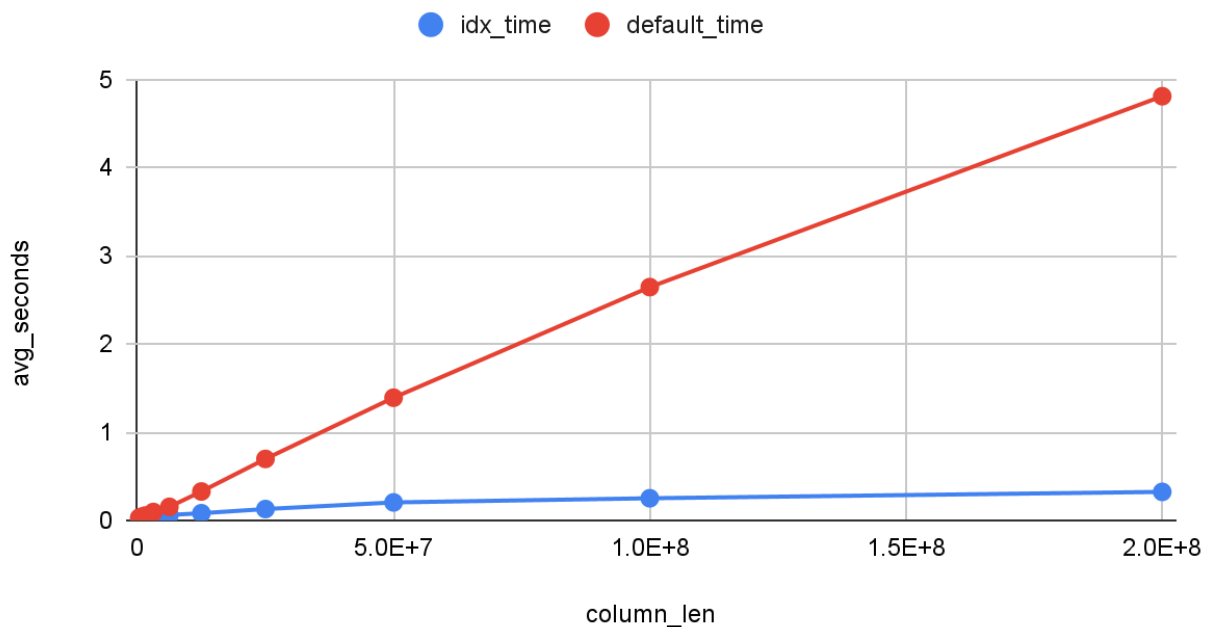
Script: https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m3e2_3.py

The purpose of this experiment is to demonstrate how selects and fetches on a clustered sorted column index scale relative to a normal column scan.

This test is a variation of M1E1. Using the same tables, I modified the setup code to add a clustered index to one of the two columns. Each test runs five (select colA, fetch colB) query pairs with random ranges and 0.5 percent selectivity. A test consists of starting the database, purging caches, running the queries where colA is either the index or non-index column, and collecting timing and page fault stats. These results are averaged across five trials and written to a CSV.

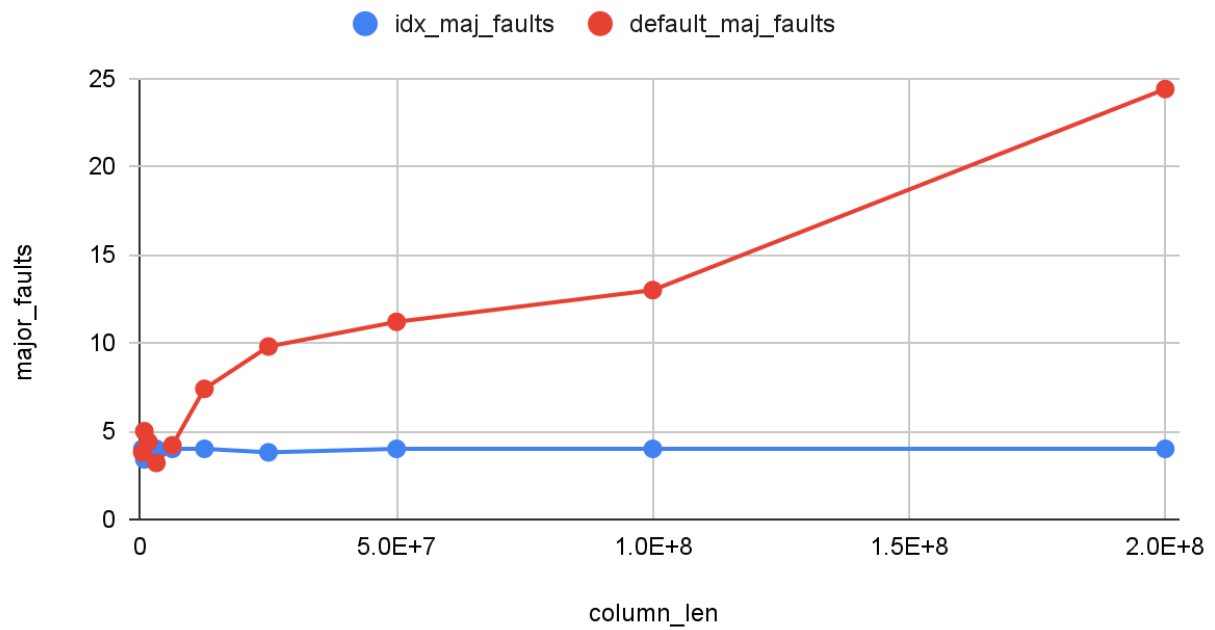
Result graph

Scan time: sorted primary v. default

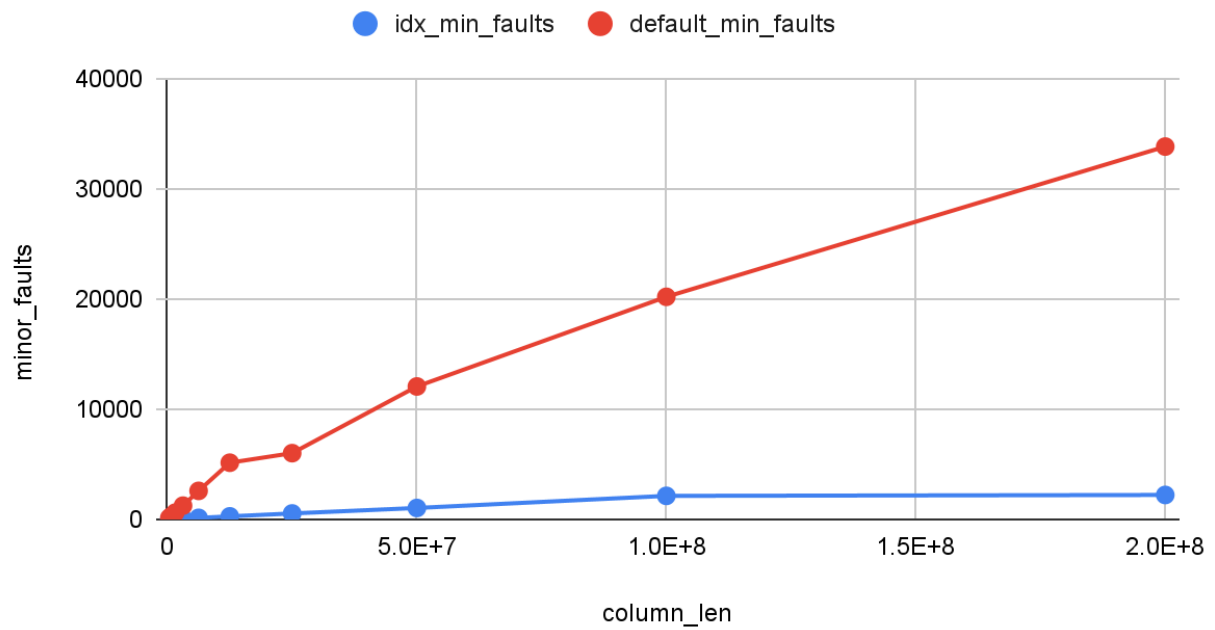


Explanation graphs

Major page faults



Minor page faults



Discussion of the result graph

This test is a variation on M1E1, so it's reassuring that the normal column scan performance looks similar. M3E1 indicates that the chosen selectivity is biased towards index performance, and this test certainly proves the worth of indexes in such circumstances. Overall, the two lines clearly compare $O(n)$ and $O(\log n)$ algorithms.

The index is also more memory friendly. At startup, a deduplicated copy of the indexed column and its positions is sorted and kept in memory. Index selection runs binary search on the in-memory copy for the start and end indices and then stores them as a "slice" selection output. When fetch sees a "slice", it just runs memcpy on the range to collect output. This is both cache and cpu efficient compared to a full table scan that touches every page in the column and iteratively builds a bitvec output.

Discussion of the explanation graphs

The index select and fetch algorithms don't really cause major page faults. The four major faults are likely from memcpy on the fetched column. Rising major faults in a table scan are expected because the scanning select operator has to visit an increasing number of column pages.

Minor page faults are more subtle, but I suspect they're entirely due to generating output. The select operator instantiates a bitvec that's 1/64 the size of a column, so memory pressure storing results grows as column size increases. Slice output from index selection, however, requires a constant 16 bytes regardless of the column length, so selections don't add memory overhead as columns grow. The only sources of memory pressure as the indexed column grows are fetch output and the index itself, which explains the small growth in minor faults as column size increases.

Experiment 3: How does a clustered B+ tree index compare to a clustered sorted index as data size grows?

Setup

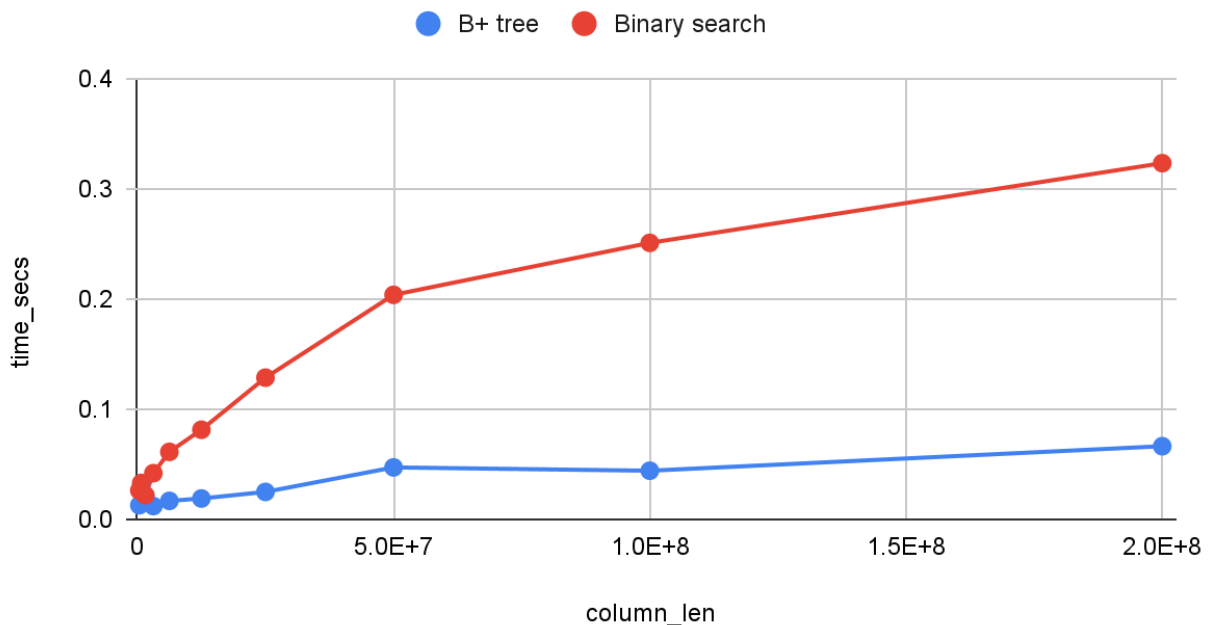
Script: https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m3e2_3.py

The purpose of this experiment is to demonstrate how selects and fetches on a clustered btree column index scale relative to a clustered sorted index.

My laptop doesn't have enough memory to load two clustered indexes of this size to compare btree versus sorted in a single test. So, I just re-ran M3E2 with a clustered btree index instead of a clustered sorted index. The tests are otherwise identical, and I ran them sequentially. Machine factors might make the timing results slightly unreliable, but I believe overall this is a fair way to compare the two index types.

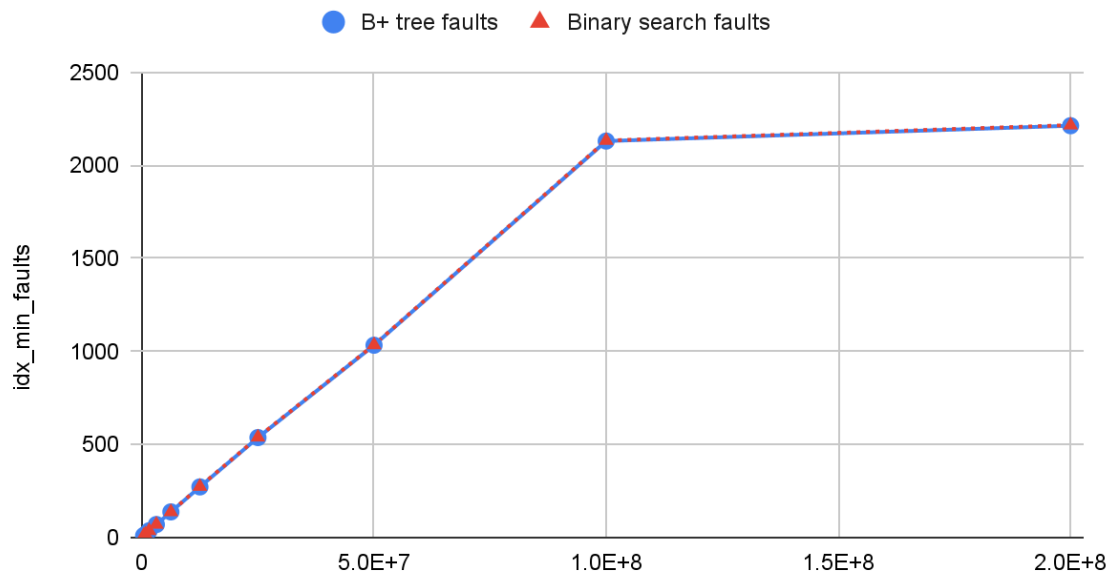
Result graph

Clustered index scan time: B+ tree v. Binary search



Explanation graph

Clustered index minor page faults: B+ tree v. Binary search



Discussion of the result graph

M3E2 showed that the sorted column index selection was already much faster than the full table scan at 0.5 percent selectivity, so the performance benefit of B+ tree over binary search is even more impressive. However, the B+ tree's flatter growth curve compared to binary search carefully matches the intuition we discussed in class. My clustered index B+ tree has a node size of 128 bytes and fanout of 10. The performance curves look convincingly like $\log_2(n)$ versus $\log_{10}(n)$, making the results seem both reasonable and satisfying (because this B+ tree warrants its complexity, unlike the B+ tree used for secondary indexes in M3E1).

Discussion of the explanation graph

This unfortunately turned into an anti-explanation graph. The major and minor page fault results for B+ tree and binary search clustered indexes are practically identical. Both index methods use the same slice output, so the page faults are likely exclusively from the fetch operator and index itself (as assumed in M3E2). I suspect one of two causes: (1) the performance difference comes from cache behavior above main memory, or (2) the algorithm is CPU bound and the B+ tree performs less computation than binary search.

At a data size of 200 million, the difference between $\log_2(200000000) = 28$ and $\log_{10}(200000000) = 9$ seems too small to explain a CPU performance difference this large, so I'm assuming hypothesis (1). The B+ tree is designed to be cache efficient with L1 and L2, so it's reasonable that performing $\sim 1/3$ the random memory accesses of the binary search is friendlier to the smaller caches unable to store the whole column.

While parsing and graphing cachegrind output for multiple trials in each data size is prohibitively time intensive, I created an additional script to validate this hypothesis. For the sorted and B+ tree primary indexes, it starts the server with cachegrind, loads the table with 200 million tuples, and runs 64 select queries:

<https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m3e3cg.py>

Even though cachegrind also includes the extra index loading work the B+ tree does on startup, output still confirms that the B+ tree suffers substantially fewer misses in D1 and LLd than the sorted index (instruction misses included in LL are less relevant here). This aligns with theoretical expectations and explains the runtime difference. Again this doesn't show up in minor page faults because the difference is largely isolated to caches above main memory.

Clustered sorted index:

```
==932==
==932== I   refs:      62,800,760,341
==932== I1  misses:      6,576
==932== LLi misses:      5,496
==932== I1  miss rate:      0.00%
==932== LLi miss rate:      0.00%
==932==
==932== D   refs:      10,216,283,046 (9,111,496,140 rd + 1,104,786,906 wr)
==932== D1  misses:      720,037,095 ( 597,744,323 rd + 122,292,772 wr)
==932== LLd misses:      719,993,760 ( 597,742,200 rd + 122,251,560 wr)
==932== D1  miss rate:      7.0% ( 6.6% + 11.1% )
==932== LLd miss rate:      7.0% ( 6.6% + 11.1% )
==932==
==932== LL refs:      720,043,671 ( 597,750,899 rd + 122,292,772 wr)
==932== LL misses:      719,999,256 ( 597,747,696 rd + 122,251,560 wr)
==932== LL miss rate:      1.0% ( 0.8% + 11.1% )
```

Clustered B+ tree index:

```
==741==
==741== I   refs:      23,880,759,905
==741== I1  misses:      7,695
==741== LLi misses:      3,979
==741== I1  miss rate:      0.00%
==741== LLi miss rate:      0.00%
==741==
==741== D   refs:      9,671,223,733 (5,584,643,117 rd + 4,086,580,616 wr)
==741== D1  misses:      449,613,138 ( 253,063,940 rd + 196,549,198 wr)
==741== LLd misses:      444,372,971 ( 247,992,895 rd + 196,380,076 wr)
==741== D1  miss rate:      4.6% ( 4.5% + 4.8% )
==741== LLd miss rate:      4.6% ( 4.4% + 4.8% )
==741==
==741== LL refs:      449,620,833 ( 253,071,635 rd + 196,549,198 wr)
==741== LL misses:      444,376,950 ( 247,996,874 rd + 196,380,076 wr)
==741== LL miss rate:      1.3% ( 0.8% + 4.8% )
```

Milestone 4

Experiment 1: How does input size affect join performance?

Setup

Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m4e1.py>

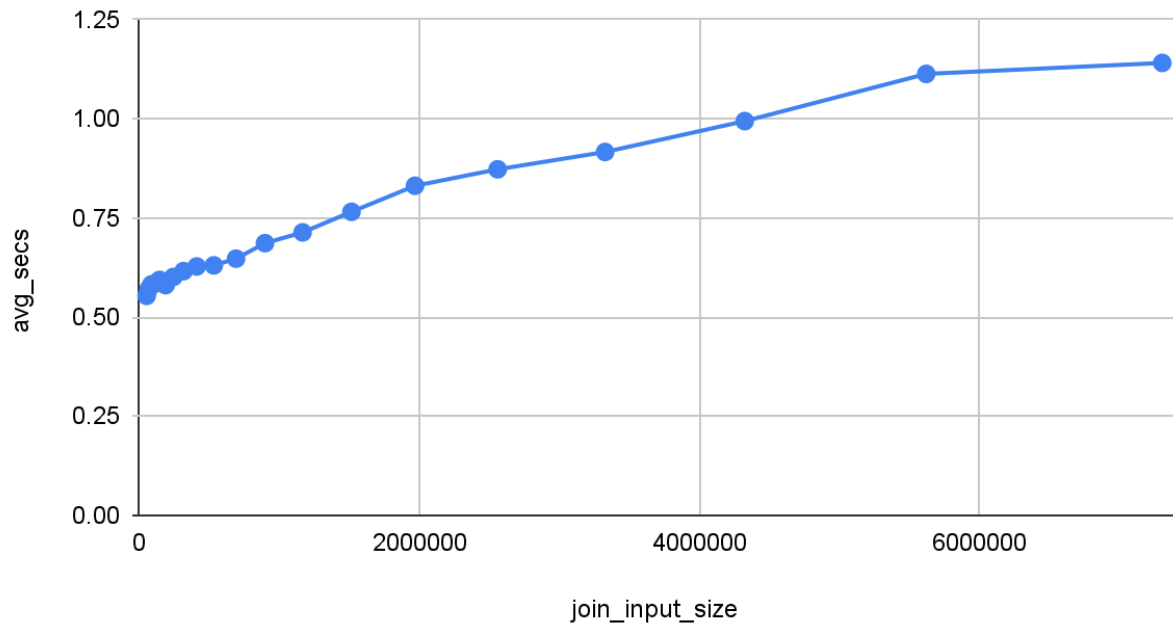
This experiment measures how hash join performance scales with input size.

This experiment uses only the two-column table with 12.5 million tuples from M1E1. For each selectivity in the range (selectivity = 0.002, selectivity < 0.36, selectivity *= 1.3), the test restarts the server and purges caches as in M1E2. It generates five groups of (select, select, fetch, fetch, join) queries with random ranges and runs them against the server, measuring completion time and server page faults. Results are averaged over five trials and written to a CSV.

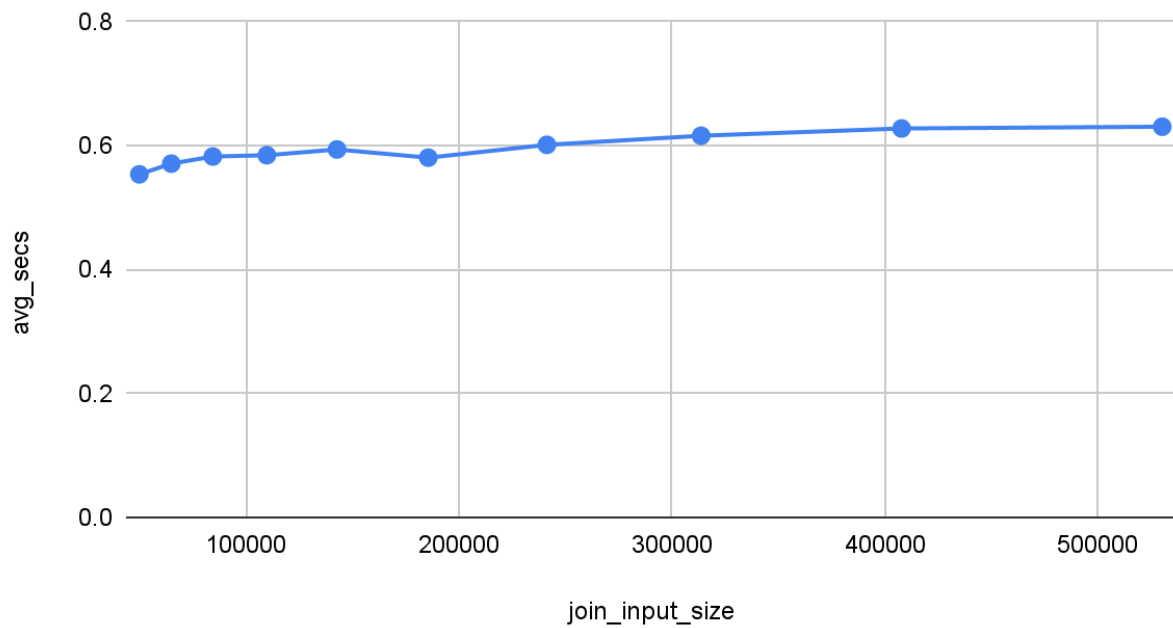
The queries only use the "hash" command for join, which allows the server to decide whether to use a single-pass or grace hash join. The charts below display an estimated input size based on selectivity, which is just the expected length of the outer and inner inputs added together: $(\text{column_len} * \text{selectivity} * 2)$.

Result graphs

Join seconds by input size (full)

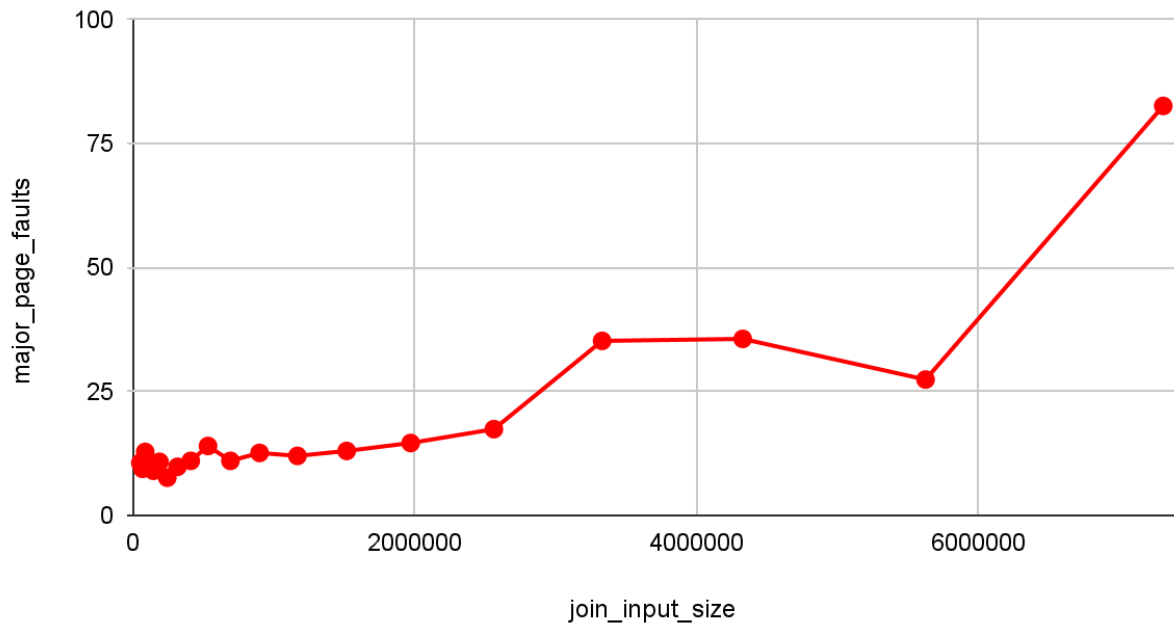


Join seconds by input size (zoomed)

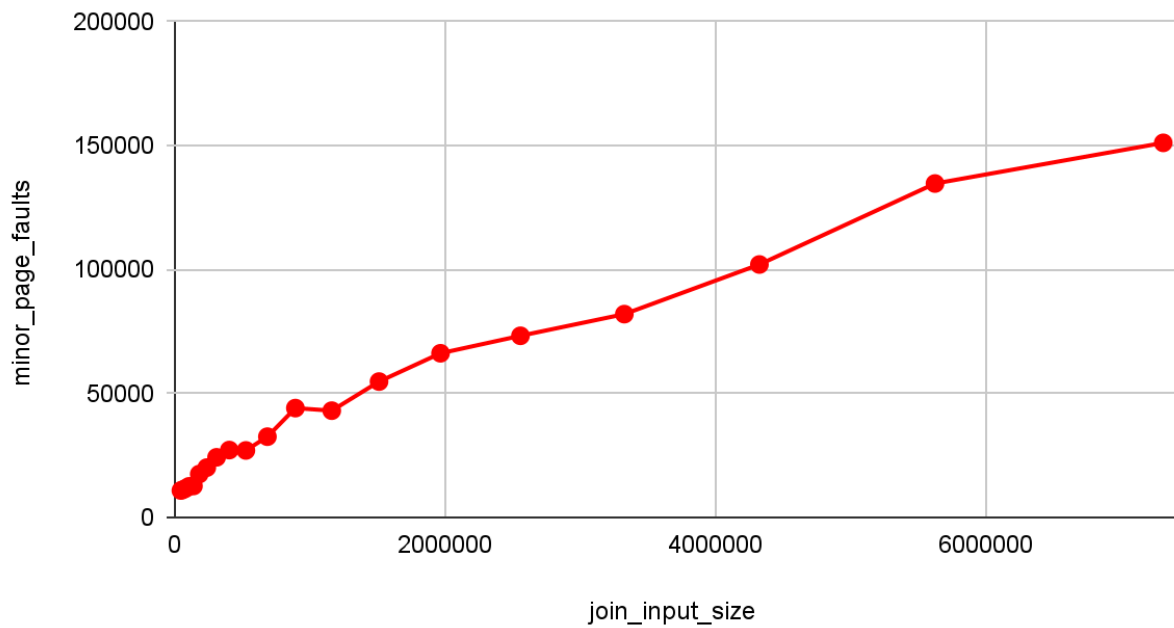


Explanation graphs

Major page faults by input size



Minor page faults by input size



Discussion of the result graphs

The results show that join runtimes grow on a gentle linear curve as input sizes increase. The rate of change in the curve is relatively low and better than I expected. The second result graph shows the clustered left side of the first graph. My decision-join algorithm switches from single-pass to grace joins once ($\text{outer_input_len} + \text{inner_input_len} \geq 174,762$), which is exactly the point at which join runtimes go down slightly in the zoomed graph. (174,762 is the number of entries in a grace partition block on disk, which takes up $4096 * 512$ bytes in total).

While single pass hash joins stay in-memory and grace joins use disk, grace joins are also parallelized across all threads in my threadpool. As soon as the single-pass hash table gets too big for comfortable use in memory, switching to grace joins with slower disk access amortized across multiple cores makes overall degradation much smoother.

Discussion of the explanation graphs

All queries have a baseline number of page faults from their selects and fetches, which explains why single pass hash joins have major page faults as well. The line of major page faults dipping and spiking in the last two tests is unexpected, although the overall trend confirms that grace joins do suffer from needing to keep partitions on disk. The relative flatness of the runtime line compared to the major faults line is likely explained by these major page faults being spread across multiple worker threads. The minor faults line is much more linear and likely reflects the memory thrashing from creating and dropping a new hash table for each grace partition. If I needed to further optimize my hash joins, I'd make a memory pool to keep and reuse a single hash table for each worker thread, which would reduce memory allocations and likely keep more of the table's metadata and buckets in L1 or L2 cache.

Experiment 2: How is join performance affected by multithreading?

Setup

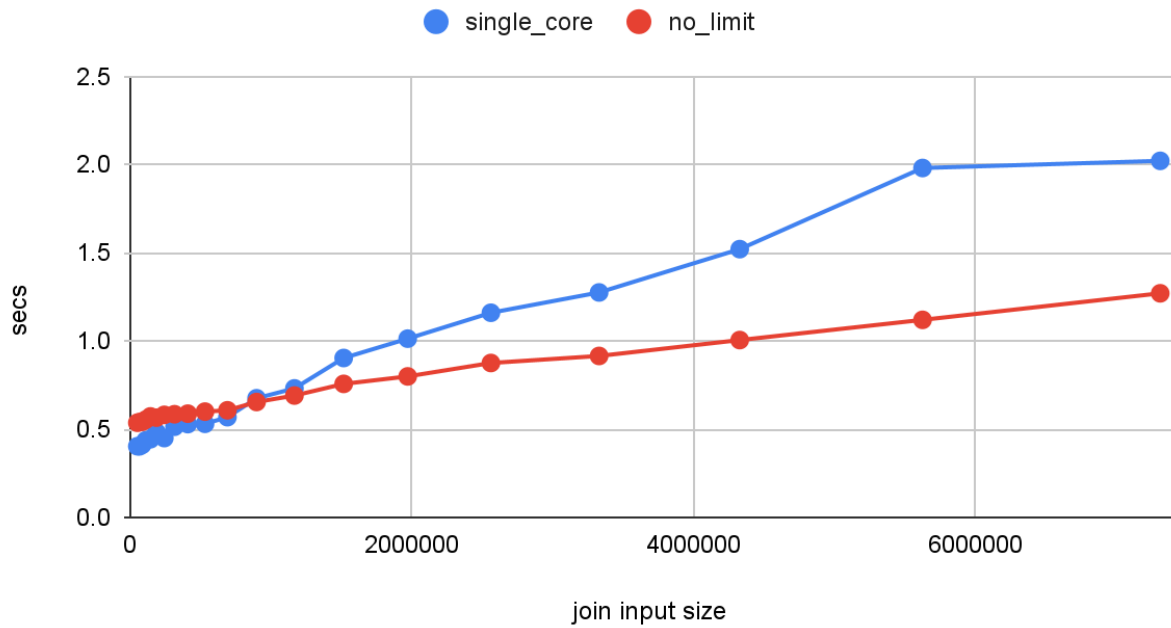
Script: <https://code.harvard.edu/coz849/cs165-project/blob/master/experiments/m4e2.py>

The goal of this test is to measure how effectively multithreading improves hash join performance.

This test uses the same queries and structure as M4E1 with the same methods for starting a single threaded server and monitoring percent CPU usage as M2E2. For each server mode and selectivity in the range (selectivity = 0.002, selectivity < 0.36, selectivity *= 1.3), the test runs five trials of five joins over the 12.5 million tuple column while recording runtime and max percent CPU usage. Results are written to a CSV.

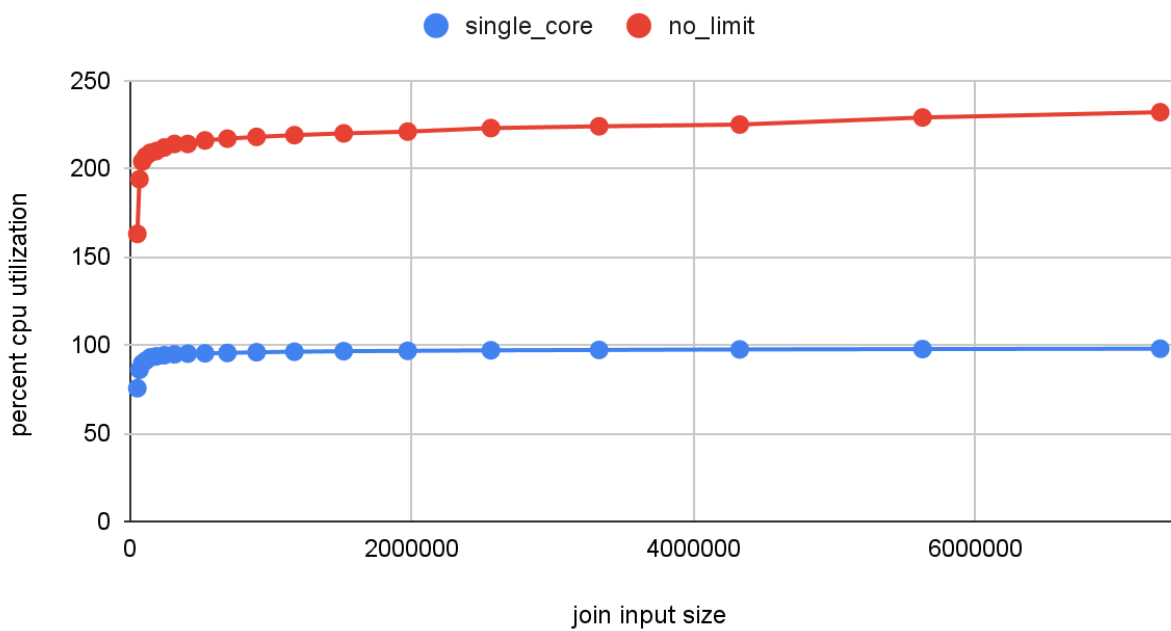
Result graph

Join runtime: single v. multi-core by input size



Explanation graph

Max PCPU per join by input size



Discussion of the result graph

Single threaded joins are a bit faster initially but are beaten substantially by the multithreaded version as input sizes grow. Multithreaded joins also seem to scale more robustly with a straighter growth curve. Recall from M4E1 that my decision-join algorithm uses single-threaded single-pass hash joins until a cutoff input size is reached, after which multithreaded grace joins are used. Higher server thread overhead might explain why the multithreaded version is slower for smaller single pass hash joins. For grace joins, however, each grace partition is joined in its own thread, giving the multithreaded version a serious advantage. While I expected the multithreaded version to be faster, I was surprised that it's also smoother. My guess is that amortizing memory movement across multiple threads leaves less risk of an outlier event slowing down a join.

Discussion of the explanation graph

The curves show the multithreaded version mirroring the single threaded version but with about 120 percent more CPU utilization. Polling pcpu leaves much room for error, so I'm pleased the curves are so well shaped. Considering the single threaded curve, small input sizes resulting in less utilization is expected. I'm pleased that utilization gets so close to 100 percent because it indicates that at least some parts of my algorithms are consistently not memory bound.

That said, I expected the multithreaded CPU utilization to reach much higher. In the last test, the input size should be roughly 7,309,602 column positions. Each partition is expected to take about $174,762 / 2 = 87,381$ entries, so there should be about $7,309,602 / 87,381 = 84$ partitions in this join. At 84 partitions, every thread should be busy joining a partition most of the time. The threadpool has nine threads on my laptop, so I expected max utilization closer to 700 (admitting synchronization costs and other machine processes). This concerns me, and increasing CPU utilization would be a good place to start if trying to optimize large joins in my DB.